

LAPORAN PRATIUM STRUKTUR DATA

IMPLEMENTASI ArrayList DAN ADT DALAM JAVA

DI SUSUN OLEH :

ABDUL KARIM ALGAZALI

NIM 2511532029

DOSEN PENGAMPU : Dr. WAHYUDI, S.T, M.T

ASISTEN LABORATORIUM : Zahran Azaria Anvaya



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa, karena atas rahmat dan izin-Nya laporan praktikum "Implementasi ArrayList dan Abstract Data Type (ADT) dalam Java" ini dapat diselesaikan dengan baik. Saya ucapkan terima kasih sebesar-besarnya kepada Dr. Wahyudi, S.T, M.T selaku dosen pengampu mata kuliah Struktur Data, serta kepada asisten laboratorium yang telah membimbing pelaksanaan praktikum ini.

Laporan ini disusun sebagai bentuk dokumentasi kegiatan praktikum yang bertujuan untuk memahami konsep penggunaan ArrayList dalam bahasa Java serta penerapan prinsip Abstract Data Type (ADT) melalui implementasi kelas DaftarKata dan Mahasiswa. Saya menyadari bahwa pemahaman tentang struktur data dinamis merupakan fondasi penting dalam pengembangan perangkat lunak yang efisien dan skalabel.

Saya menyadari bahwa penulisan laporan praktikum ini masih jauh dari kata sempurna. Namun, dengan segala kemampuan dan pengetahuan yang dimiliki, saya telah berupaya supaya laporan ini dapat selesai dengan baik. Oleh karenanya, saya dengan rendah hati menerima saran, masukan, dan kritikan guna penyempurnaan laporan ini.

Padang, April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang	3
1.2 Tujuan	3
1.3 Manfaat	3
BAB II PEMBAHASAN.....	4
2.1 Konsep ArrayList dalam Java.....	4
2.2 Langkah pengerjaan	5
BAB III KESIMPULAN.....	13
3.1 Kesimpulan	13
3.2 Saran	13
DAFTAR PUSTAKA.....	14

BAB I

PENDAHULUAN

1.1 Latar Belakang

ArrayList adalah implementasi dari interface List dalam Java Collection Framework yang memungkinkan penyimpanan kumpulan objek dengan ukuran dinamis. Berbeda dengan array konvensional yang ukurannya tetap, ArrayList dapat bertambah atau berkurang secara otomatis sesuai kebutuhan.

Dalam praktikum ini, peserta mengimplementasikan lima kelas Java untuk memahami:

1. Operasi dasar [ArrayList](#) (penambahan, penghapusan, akses elemen)
2. Prinsip enkapsulasi melalui pembuatan kelas ADT [DaftarKata](#)
3. Penerapan [ArrayList](#) dalam manajemen data objek [Mahasiswa](#) dengan fitur CRUD (Create, Read, Update, Delete)

1.1 Tujuan

1. Memahami cara kerja dan operasi dasar ArrayList dalam Java.
2. Mampu membuat kelas ADT yang mengenkapsulasi ArrayList dengan metode yang terstruktur.
3. Mengimplementasikan program interaktif dengan menu untuk manajemen data menggunakan ArrayList.
4. Melatih keterampilan pemrograman berorientasi objek dengan prinsip enkapsulasi dan modularitas.

1.2 Manfaat

1. Mahasiswa memahami perbedaan antara array statis dan ArrayList dinamis.
2. Mahasiswa mampu merancang kelas yang memisahkan antarmuka publik dari implementasi internal.
3. Mahasiswa memperoleh pengalaman dalam membuat program interaktif dengan validasi input dan struktur menu.

BAB II

PEMBAHASAN

2.1 Konsep ArrayList dalam Java

ArrayList merupakan bagian dari Java Collections Framework yang menyediakan:

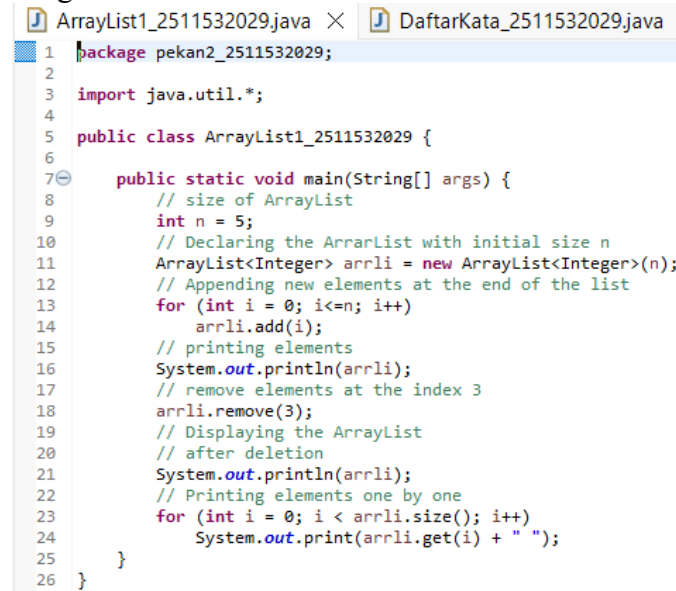
1. Ukuran dinamis (auto-resizing)
2. Akses acak (random access) dengan kompleksitas $O(1)$ untuk `get(index)`
3. Metode utilitas: `add()`, `remove()`, `contains()`, `size()`, `clear()`, dll.

2.2 Langkah pengerjaan

1. Pembuatan kelas ArrayList1_2511532029

Kelas ini mendemonstrasikan operasi fundamental ArrayList<Integer>

Program:



```
1 package pekan2_2511532029;
2
3 import java.util.*;
4
5 public class ArrayList1_2511532029 {
6
7     public static void main(String[] args) {
8         // size of ArrayList
9         int n = 5;
10        // Declaring the ArrayList with initial size n
11        ArrayList<Integer> arrli = new ArrayList<Integer>(n);
12        // Appending new elements at the end of the list
13        for (int i = 0; i <= n; i++)
14            arrli.add(i);
15        // printing elements
16        System.out.println(arrli);
17        // remove elements at the index 3
18        arrli.remove(3);
19        // Displaying the ArrayList
20        // after deletion
21        System.out.println(arrli);
22        // Printing elements one by one
23        for (int i = 0; i < arrli.size(); i++)
24            System.out.print(arrli.get(i) + " ");
25    }
26 }
```

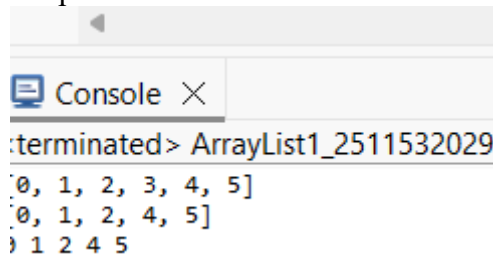
Gambar 2.1

Penjelasan program:

- 1) `int n = 5;`
Mendeklarasikan variabel integer n dengan nilai 5 yang akan digunakan sebagai acuan ukuran awal ArrayList.
- 2) `ArrayList<Integer> arrli = new ArrayList<Integer>(n);`
Membuat objek ArrayList dengan tipe data Integer dan kapasitas awal 5. Kapasitas ini bersifat fleksibel, ArrayList dapat tumbuh otomatis saat elemen ditambahkan melebihi kapasitas awal.
- 3) `for (int i = 0; i <= n; i++) arrli.add(i);`
Perulangan untuk menambahkan elemen bernilai 0 sampai 5 ke dalam ArrayList menggunakan metode `add()` yang menyisipkan elemen di akhir list. Total elemen yang ditambahkan adalah 6 buah (0,1,2,3,4,5).
- 4) `System.out.println(arrli);`
Mencetak seluruh isi ArrayList. Metode `toString()` dari ArrayList otomatis dipanggil sehingga output ditampilkan dalam format `[0, 1, 2, 3, 4, 5]`.
- 5) `arrli.remove(3);`
Menghapus elemen pada indeks 3 (bukan nilai 3). Elemen yang terhapus adalah angka 3, dan elemen setelahnya (4 dan 5) akan bergeser ke kiri mengisi posisi yang kosong.
- 6) `System.out.println(arrli);`
Mencetak isi ArrayList setelah penghapusan. Output yang dihasilkan adalah `[0, 1, 2, 4, 5]`.

- 7) `for (int i = 0; i < arrli.size(); i++) System.out.print(arrli.get(i) + " ");`
 Iterasi manual menggunakan indeks untuk mengakses elemen satu per satu dengan metode `get(i)`. Setiap elemen dicetak diikuti spasi tanpa baris baru.
- o. `arrli.size()` mengembalikan jumlah elemen saat ini yaitu 5.
 - o. `arrli.get(i)` mengambil nilai pada indeks ke-i.
 - o. Output: 0 1 2 4 5

Output :



Gambar 2.2

2. Pembuatan kelas DaftarKata_2511532029

Program:

```

1 package pekan2_2511532029;
2 import java.util.ArrayList;
3
4 public class DaftarKata_2511532029 {
5
6     private final ArrayList<String> data;
7
8     // konstruktor: inisialisasi list kosong
9     public DaftarKata_2511532029() {
10         this.data = new ArrayList<>();
11     }
12
13     /** menambahkan element di akhir list. */
14     public void tambah_2511532029(String elemen) {
15         data.add(elemen);
16     }
17
18     /** Menambahkan elemen pada indeks tertentu (menyisipkan) */
19     public void tambahPada_2511532029(int index, String elemen) {
20         data.add(index, elemen);
21     }
22
23     /**
24      * Mengubah elemen pada posisi 'index' menjadi 'nilai baru'
25      * Bertindak sebagai "setter" untuk elemen tertentu.
26      */
27     public void ubahElemen_2511532029(int index, String nilaiBaru) {
28         data.set(index, nilaiBaru);
29     }
30
31     /**
32      * Menghapus elemen pada posisi 'index' dan mengembalikan nilai yang dihapus
33      */
34     public String hapusElemen_2511532029(int index) {
35         return data.remove(index);
36     }
37
38     /**
39      * Melakukan iterasi dan mencetak setiap elemen format : [index] nilai
40      * (Metode ini tidak mengembalikan nilai: hanya demonstrasi iterasi)
41      */
42     public void iterasiCetak_2511532029() {
43         for (int i = 0; i < data.size(); i++) {
44             System.out.println("[ " + i + " ] " + data.get(i));
45         }
46     }
47
48     public String get(int index) {
49         return data.get(index);
50     }
51
52     @Override
53     public String toString() {
54         return data.toString();
55     }
56 }

```

Gambar 2.3

Penjelasan program:

- 1) `private final ArrayList<String> data;` Mendeklarasikan atribut privat bertipe `ArrayList<String>` dengan modifier final. Modifier final berarti referensi objek tidak dapat diubah setelah inisialisasi, menerapkan prinsip enkapsulasi penuh.
- 2) `public DaftarKata_2511532029() { this.data = new ArrayList<>(); }` Konstruktor tanpa parameter yang menginisialisasi data sebagai `ArrayList` kosong saat objek dibuat.
- 3) `public void tambah_2511532029(String elemen) { data.add(elemen); }` Metode publik untuk menambahkan elemen string di **akhir** list (operasi append). Metode ini tidak mengembalikan nilai (void).
- 4) `public void tambahPada_2511532029(int index, String elemen) { data.add(index, elemen); }` Metode untuk menyisipkan elemen string pada **indeks tertentu**. Elemen yang sudah ada pada indeks tersebut dan setelahnya akan bergeser ke kanan.
- 5) `public void ubahElemen_2511532029(int index, String nilaiBaru) { data.set(index, nilaiBaru); }` Metode "setter" untuk mengganti nilai pada indeks tertentu dengan nilai baru tanpa mengubah ukuran list.
- 6) `public String hapusElemen_2511532029(int index) { return data.remove(index); }` Menghapus elemen pada indeks yang ditentukan dan mengembalikan nilai yang dihapus bertipe String untuk keperluan logging atau validasi.
- 7) `public void iterasiCetak_2511532029() { ... }` Metode traversal yang mencetak setiap elemen dengan format `[index] nilai` menggunakan perulangan for tradisional untuk demonstrasi iterasi manual.
 - o. `for (int i = 0; i < data.size(); i++)` mengiterasi dari indeks 0 hingga ukuran list.
 - o. `System.out.println("[ " + i + " ] " + data.get(i));` mencetak format terstruktur per elemen.
- 8) `public String toString() { return data.toString(); }` Override metode `toString()` untuk representasi string yang mudah dicetak, memanfaatkan implementasi `toString()` dari `ArrayList` yang sudah terformat rapi.

3. Pembuatan kelas DaftarKata_2511532029

Program:



```
1 package pekan2_2511532029;
2
3 public class DaftarKataDriver_2511532029 {
4
5     public static void main(String[] args) {
6         DaftarKata_2511532029 al = new DaftarKata_2511532029();
7
8         // Add element
9         al.tambah_2511532029("Kami");
10        al.tambah_2511532029("Informatika");
11
12        // Insert element
13        al.tambahPada_2511532029(1, "Mahasiswa");
14
15        // Print
16        System.out.printf("Awal          : %s\n", al);
17
18        // Change element (index 1)
19        al.ubahElemen_2511532029(1, "Departemen");
20        System.out.printf("Setelah Ubah : %s\n", al);
21
22        // Delete
23        String terhapus = al.hapusElemen_2511532029(0);
24        System.out.printf("Terhapus    : %s\n", terhapus);
25        System.out.printf("Setelah Hapus : %s\n", al);
26
27        // Iteration
28        System.out.print("Iterasi    : ");
29        al.iterasiCetak_2511532029();
30        System.out.println();
31    }
32 }
33 }
```

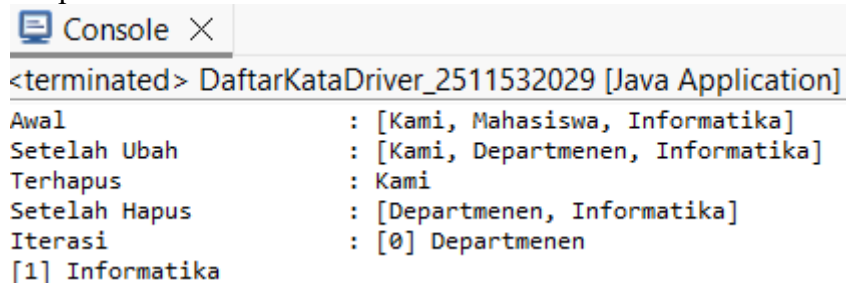
Gambar 2.4

Penjelasan program:

- 1) `DaftarKata_2511532029 al = new DaftarKata_2511532029();`
Membuat objek ADT DaftarKata yang siap digunakan untuk operasi manipulasi data string.
- 2) `al.tambah_2511532029("Kami");`
Menambahkan string "Kami" di akhir list. Isi list sekarang: [Kami].
- 3) `al.tambah_2511532029("Informatika");`
Menambahkan string "Informatika" di akhir list. Isi list sekarang: [Kami, Informatika].
- 4) `al.tambahPada_2511532029(1, "Mahasiswa");`
Menyisipkan string "Mahasiswa" pada indeks 1. Elemen "Informatika" bergeser ke indeks 2.
o. Isi list sekarang: [Kami, Mahasiswa, Informatika].
- 5) `System.out.printf("Awal: %s\n", al);`
Mencetak isi list awal menggunakan format printf. Metode `toString()` otomatis dipanggil.
o. Output: Awal: [Kami, Mahasiswa, Informatika].
- 6) `al.ubahElemen_2511532029(1, "Departemen");`
Mengganti elemen pada indeks 1 dari "Mahasiswa" menjadi "Departemen".
o. Isi list sekarang: [Kami, Departemen, Informatika].
- 7) `System.out.printf("Setelah Ubah: %s\n", al);`
Mencetak isi list setelah perubahan.
o. Output: Setelah Ubah: [Kami, Departemen, Informatika].
- 8) `String terhapus = al.hapusElemen_2511532029(0);`
Menghapus elemen pada indeks 0 ("Kami") dan menyimpan nilai yang dihapus dalam variabel `terhapus`.
o. Isi list sekarang: [Departemen, Informatika].
- 9) `System.out.printf("Terhapus: %s\n", terhapus);`
Menampilkan nilai yang berhasil dihapus.
o. Output: Terhapus: Kami.

- 10) `System.out.printf("Setelah Hapus: %s%n", al);`
Mencetak isi list setelah penghapusan.
o. Output: Setelah Hapus: [Departemen, Informatika].
- 11) `al.iterasiCetak_2511532029();`
Memanggil metode iterasi untuk mencetak elemen satu per satu dengan format terstruktur.

Output:

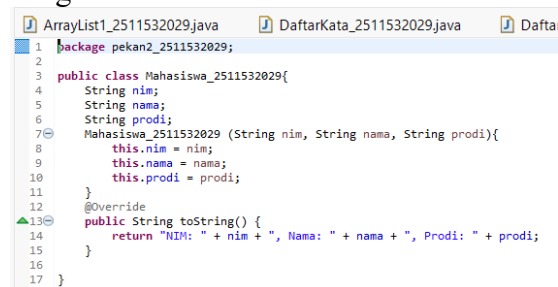


```
<terminated> DaftarKataDriver_2511532029 [Java Application]
Awal : [Kami, Mahasiswa, Informatika]
Setelah Ubah : [Kami, Departemenen, Informatika]
Terhapus : Kami
Setelah Hapus : [Departemenen, Informatika]
Iterasi : [0] Departemenen
[1] Informatika
```

Gambar 2.5

4. Pembuatan kelas Mahasiswa_2511532029

Program:



```
1 package pekan2_2511532029;
2
3 public class Mahasiswa_2511532029{
4     String nim;
5     String nama;
6     String prodi;
7     Mahasiswa_2511532029 (String nim, String nama, String prodi){
8         this.nim = nim;
9         this.nama = nama;
10        this.prodi = prodi;
11    }
12    @Override
13    public String toString() {
14        return "NIM: " + nim + ", Nama: " + nama + ", Prodi: " + prodi;
15    }
16 }
17 }
```

Gambar 2.6

Penjelasan program:

- 1) `String nim; String nama; String prodi;`
Mendeklarasikan tiga atribut bertipe String untuk merepresentasikan data identitas mahasiswa: Nomor Induk Mahasiswa, nama lengkap, dan program studi.
o Catatan: Untuk enkapsulasi optimal, atribut ini sebaiknya diberi modifier `private`.
- 2) `Mahasiswa_2511532029(String nim, String nama, String prodi) { ... }`
Konstruktor berparameter untuk inisialisasi objek mahasiswa secara lengkap saat pembuatan.
o. `this.nim = nim;` menetapkan nilai parameter `nim` ke atribut objek.
o. `this.nama = nama;` menetapkan nilai parameter `nama` ke atribut objek.
o. `this.prodi = prodi;` menetapkan nilai parameter `prodi` ke atribut objek.
- 3) `@Override public String toString() { return "NIM: " + nim + ", Nama: " + nama + ", Prodi: " + prodi; }`
Override metode `toString()` untuk format output yang rapi saat objek dicetak.
o. Memudahkan debugging dan display data tanpa perlu memanggil getter satu per satu.
o. Output contoh: NIM: 2511532029, Nama: Abdul Karim, Prodi: Informatika.

5. Pembuatan kelas MahasiswaDriver_2511532029

Program:

```

1 package pekan2_2511532029;
2 import java.util.*;
3 public class MahasiswaDriver_2511532029 {
4     // 1. Method untuk menampilkan menu
5     public static void tampilMenu_2511532029() {
6         System.out.println("Menu: ");
7         System.out.println("1. Tambah Mahasiswa");
8         System.out.println("2. Tampilkan Semua Mahasiswa");
9         System.out.println("3. Hapus Mahasiswa Berdasarkan NIM");
10        System.out.println("4. Cari Mahasiswa Berdasarkan NIM");
11        System.out.println("5. Keluar");
12    }
13
14    // 2. Method untuk tambah mahasiswa
15    public static void tambahMahasiswa_2511532029 (ArrayList<Mahasiswa_2511532029> list, Scanner sc) {
16        System.out.println("Masukan NIM : ");
17        String nim = sc.nextLine();
18        System.out.println("Masukan Nama : ");
19        String nama = sc.nextLine();
20        System.out.println("Masukan Prodi : ");
21        String prodi = sc.nextLine();
22        list.add(new Mahasiswa_2511532029 (nim, nama, prodi));
23        System.out.println("Mahasiswa berhasil ditambahkan");
24    }
25
26    // 3. Method untuk tampilkan semua data
27    public static void tampilSemuaMahasiswa_2511532029 (ArrayList<Mahasiswa_2511532029> list) {
28        if (list.isEmpty()) {
29            System.out.println("Daftar mahasiswa kosong");
30        } else {
31            System.out.println("Data mahasiswa :");
32            for (Mahasiswa_2511532029 mhs : list) {
33                System.out.println(mhs);
34            }
35        }
36    }
37
38    // 4. Method untuk hapus mahasiswa berdasarkan NIM
39    public static void hapusMahasiswa_2511532029 (ArrayList<Mahasiswa_2511532029> list, Scanner sc) {
40        System.out.println("Masukan NIM yang akan dihapus:");
41        String nimhapus = sc.nextLine();
42        boolean removed = list.removeIf(mhs -> mhs.nim.equals(nimhapus));
43        if (removed) {
44            System.out.println("Data dengan NIM " + nimhapus + " berhasil dihapus.");
45        } else {
46            System.out.println("NIM Tidak ditemukan.");
47        }
48    }
49
50    // 5. Method untuk cari mahasiswa berdasarkan NIM
51    public static void cariMahasiswa_2511532029 (ArrayList<Mahasiswa_2511532029> list, Scanner sc) {
52        System.out.println("Masukan NIM yang dicari:");
53        String nimcar = sc.nextLine();
54        boolean ditemukan = false;
55        for (Mahasiswa_2511532029 mhs : list) {
56            if (mhs.nim.equals(nimcar)) {
57                System.out.println("Hasil Pencarian : " + mhs);
58                ditemukan = true;
59                break;
60            }
61        }
62        if (!ditemukan) {
63            System.out.println("IDP tidak ada.");
64        }
65    }
66
67    public static void main (String[] args) {
68        ArrayList<Mahasiswa_2511532029> mahasiswaList = new ArrayList<>();
69        Scanner scanner = new Scanner(System.in);
70        int choice;
71
72        do {
73            tampilMenu_2511532029();
74            System.out.println("Pilih Menu:");
75            choice = scanner.nextInt();
76            scanner.nextLine();
77
78            switch (choice) {
79                case 1:
80                    tambahMahasiswa_2511532029(mahasiswaList, scanner);
81                    break;
82                case 2:
83                    tampilSemuaMahasiswa_2511532029(mahasiswaList);
84                    break;
85                case 3:
86                    hapusMahasiswa_2511532029(mahasiswaList, scanner);
87                    break;
88                case 4:
89                    cariMahasiswa_2511532029(mahasiswaList, scanner);
90                    break;
91                case 5:
92                    System.out.println("Keluar dari program");
93                    break;
94                default:
95                    System.out.println("Pilihan tidak valid");
96            }
97        } while (choice != 5);
98        scanner.close();
99    }
100
101 }
102
103

```

Gambar 2.7

Penjelasan Program - Metode Utama:

1. `ArrayList<Mahasiswa_2511532029> mahasiswaList = new ArrayList<>();`
Membuat ArrayList kosong untuk menyimpan objek-objek Mahasiswa sebagai struktur data utama program.
2. `Scanner scanner = new Scanner(System.in);` Membuat objek Scanner untuk membaca input dari keyboard pengguna.
3. `do { ... } while (choice != 5);` Struktur perulangan do-while yang memastikan menu ditampilkan minimal sekali dan program terus berjalan hingga pengguna memilih opsi keluar (5).

Penjelasan Program - Metode Menu dan Operasi:

4. `tampilkanMenu_2511532029();` Memanggil metode untuk menampilkan pilihan menu interaktif kepada pengguna dengan 5 opsi operasi.
5. `choice = scanner.nextInt(); scanner.nextLine();` Membaca pilihan angka dari pengguna.
o. `scanner.nextLine();` diperlukan untuk mengonsumsi karakter newline yang tersisa setelah `nextInt()`, agar input string berikutnya tidak terlewat.

6. case 1: tambahMahasiswa_2511532029(mahasiswaList, scanner); Memanggil metode untuk menambah data mahasiswa baru.
 - o. Membaca input NIM, nama, dan prodi menggunakan scanner.nextLine().
 - o. Membuat objek baru: new Mahasiswa_2511532029(nim, nama, prodi).
 - o. Menambahkan objek ke list: list.add(...).
 - o. Output konfirmasi: Mahasiswa berhasil ditambahkan.
7. case 2: tampilkanSemuaMahasiswa_2511532029(mahasiswaList); Memanggil metode untuk menampilkan seluruh data mahasiswa.
 - o if (list.isEmpty()) mengecek apakah list kosong, jika ya tampilkan pesan "Daftar mahasiswa kosong".
 - o for (Mahasiswa_2511532029 mhs : list) menggunakan enhanced for-loop untuk iterasi yang lebih ringkas.
 - o System.out.println(mhs) otomatis memanggil metode toString() dari setiap objek Mahasiswa.
8. case 3: hapusMahasiswa_2511532029(mahasiswaList, scanner); Memanggil metode untuk menghapus mahasiswa berdasarkan NIM.
 - o. String nimHapus = sc.nextLine(); membaca NIM yang akan dihapus.
 - o. list.removeIf(mhs -> mhs.nim.equals(nimHapus)) menggunakan lambda expression (Java 8+) untuk menghapus elemen yang memenuhi kondisi.
 - o. boolean removed menampung hasil operasi: true jika berhasil, false jika NIM tidak ditemukan.
 - o Output: pesan sukses atau "NIM Tidak ditemukan."
9. case 4: cariMahasiswa_2511532029(mahasiswaList, scanner); Memanggil metode untuk mencari mahasiswa berdasarkan NIM.
 - o String nimCari = sc.nextLine(); membaca NIM yang dicari.
 - o boolean ditemukan = false; flag untuk menandai status pencarian.
 - o Perulangan for (Mahasiswa_2511532029 mhs : list) melakukan linear search.
 - o if (mhs.nim.equals(nimCari)) mengecek kecocokan NIM.
 - o break; menghentikan pencarian segera setelah data ditemukan untuk efisiensi.
 - o Output: data mahasiswa jika ditemukan, atau "NIM tidak ada." jika tidak.
10. case 5: System.out.println("Keluar dari program"); Menampilkan pesan penutup dan mengakhiri perulangan karena kondisi choice != 5 menjadi false.
11. scanner.close(); Menutup objek Scanner untuk melepaskan resource sistem dan mencegah kebocoran memori.

Output:

```
Console X
MahasiswaDriver_2511532029 [Java Application] C:\Users\USER\p2\pool\plugins

Menu:
1.Tambah Mahasiswa
2.Tampilkan Semua Mahasiswa
3.Hapus Mahasiswa Berdasarkan NIM
4.Cari Mahasiswa Berdasarkan NIM
5.Keluar
Pilih Menu:
1
Masukan NIM :
2511532029
Masukan Nama :
karim
Masukan Prodi :
Informatika
Mahasiswa berhasil ditambahkan

Menu:
1.Tambah Mahasiswa
2.Tampilkan Semua Mahasiswa
3.Hapus Mahasiswa Berdasarkan NIM
4.Cari Mahasiswa Berdasarkan NIM
5.Keluar
Pilih Menu:
2
Data mahasiswa :
NIM: 2511532029, Nama: karim, Prodi: Informatika

Menu:
1.Tambah Mahasiswa
2.Tampilkan Semua Mahasiswa
3.Hapus Mahasiswa Berdasarkan NIM
4.Cari Mahasiswa Berdasarkan NIM
5.Keluar
Pilih Menu:
4
Masukan NIM yang dicari:
2511532029
Hasil Pencarian:NIM: 2511532029, Nama: karim, Prodi: Informatika

Menu:
1.Tambah Mahasiswa
2.Tampilkan Semua Mahasiswa
3.Hapus Mahasiswa Berdasarkan NIM
4.Cari Mahasiswa Berdasarkan NIM
5.Keluar
Pilih Menu:
3
Masukan NIM yang akan dihapus:
2511532029
Data dengan NIM2511532029Berhasil dihapus.

Menu:
1.Tambah Mahasiswa
2.Tampilkan Semua Mahasiswa
3.Hapus Mahasiswa Berdasarkan NIM
4.Cari Mahasiswa Berdasarkan NIM
5.Keluar
Pilih Menu:
4
Masukan NIM yang dicari:
2511532029
NIM tidak ada.

Menu:
1.Tambah Mahasiswa
2.Tampilkan Semua Mahasiswa
3.Hapus Mahasiswa Berdasarkan NIM
4.Cari Mahasiswa Berdasarkan NIM
5.Keluar
Pilih Menu:
5
Keluar dari program
```

Gambar 2.8

BAB III KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum implementasi ArrayList dan Abstract Data Type (ADT) di Java, dapat disimpulkan bahwa ArrayList merupakan struktur data dinamis yang sangat fleksibel untuk mengelola kumpulan data dengan ukuran yang berubah-ubah, berkat operasi dasarnya seperti `add()`, `remove()`, `get()`, dan `set()` yang berjalan secara intuitif. Penerapan prinsip ADT melalui kelas `DaftarKata` dan `Mahasiswa` berhasil menunjukkan pemisahan yang jelas antara antarmuka publik dan implementasi internal. Dengan memanfaatkan enkapsulasi berupa atribut `private` dan metode akses terkontrol, data terlindungi dari modifikasi tidak sah, sekaligus menciptakan kode yang modular, aman, dan mudah dipelihara. Perubahan struktur internal di kemudian hari tidak akan memengaruhi kode klien selama antarmuka metode tetap konsisten.

Secara praktis, integrasi ADT ke dalam program interaktif berbasis menu menggunakan `Scanner` dan struktur `do-while` membuktikan bahwa konsep struktur data dapat diwujudkan dalam aplikasi yang fungsional dan ramah pengguna. Penggunaan fitur Java modern seperti *enhanced for-loop* untuk iterasi yang lebih ringkas dan ekspresi lambda pada metode `removeIf()` menunjukkan efisiensi dalam penulisan kode kondisional. Selain itu, penerapan penanganan status operasi dan logika pencarian linear berhasil meningkatkan ketangguhan program. Secara keseluruhan, praktikum ini tidak hanya memperkuat pemahaman tentang manajemen data dinamis dan pemrograman berorientasi objek, tetapi juga menjadi fondasi yang kokoh untuk mempelajari struktur data yang lebih kompleks seperti `LinkedList`, `Stack`, dan `Queue` di sesi mendatang.

3.2 Saran

1. Akan lebih baik bila dosen menyelenggarakan sesi pra-praktikum di kelas agar mahasiswa dapat memperoleh pemahaman awal yang lebih memadai, sehingga dapat menghindari kepanikan atau kesalahan saat praktikum
2. Sebaiknya dosen membagikan materi praktikum terlebih dahulu melalui iLearn agar mahasiswa bisa mempersiapkan diri sebelum pelaksanaan praktikum

DAFTAR PUSTAKA

[1] Oracle. "ArrayList (Java Platform SE 17)". *The Java™ Tutorials*. 2024.

Available:

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/ArrayList.html>